```
        response.EnsureSuccessStatusCode();
        var content = await response.Content.ReadAsStringAsync();

        Assert.Equal("3", content);                                    ❹
    }
}
```

❶ Implements the IClassFixture interface for the custom factory

❷ Injects an instance of the factory in the constructor

❸ The client already contains the test service configuration.

❹ The result confirms that the test service was used.

You can also combine your custom `WebApplicationFactory`, which substitutes services that you always want to replace, with the `WithWebHostBuilder()` method to override additional services on a per-test basis. That combination gives you the best of both worlds: reduced duplication with the custom factory and control with the per-test configuration.

Running integration tests using your real app's configuration provides about the closest thing you'll get to a guarantee that your app is working correctly. The sticking point in that guarantee is nearly always external dependencies, such as third-party APIs and databases.

In the final section of this chapter we'll look at how to use the SQLite provider for EF Core with an in-memory database. You can use this approach to write tests for services that use an EF Core database context without needing access to a real database.

36.4 Isolating the database with an in-memory EF Core provider